Week-12

Introduction to Database Systems

Shah Nawaz

Indexing

Organization of Records in a File

Organization of records/data in general deals with how the records are stored and broadly the organizations can be divided into two types:

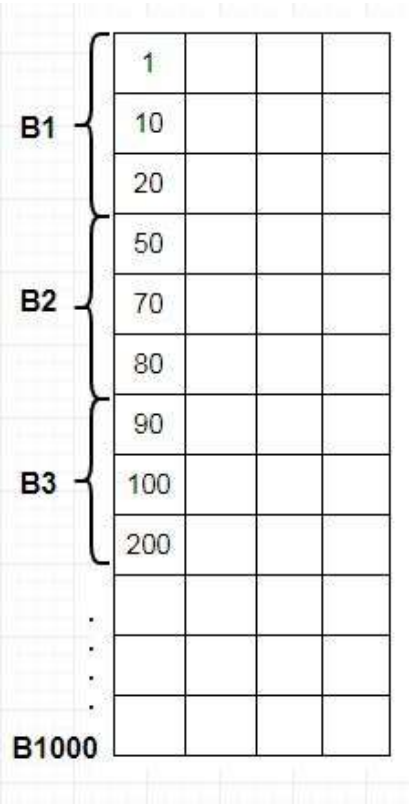
1. **Ordered organization**:
   All the records in a file are ordered in some search key value.
2. **Unordered organization**:
   In this type, records are inserted in the file wherever the place is available, usually to the very end of the file.

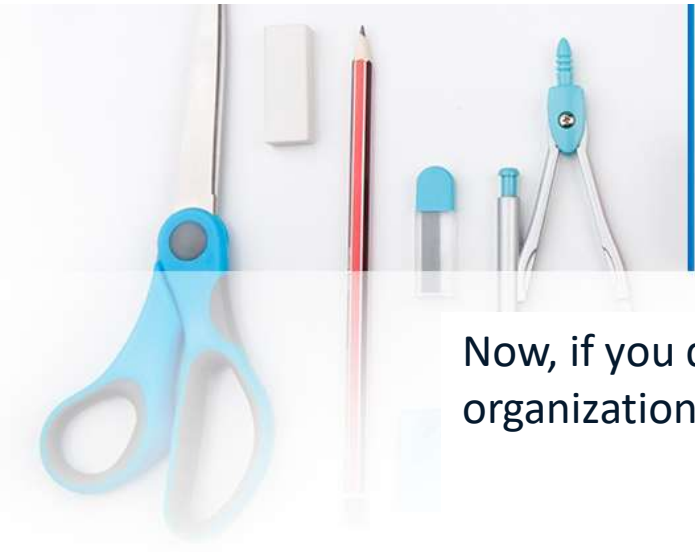
Need for Indexing

Consider you have a table (relation) consisting several records in a database. The records are further divided into 1000 blocks. Pictorially, the organization looks like given in the picture.

From this organization, it is clear that -

- The order is applied to the first column of records
- The number of blocks in which the total data is divided 1000 blocks.





Indexing explained

Now, if you do a *binary search* in order to search for anything inside this organization or records, the total time it will take is

$Log_2 1000$

- To calculate $\log_2 1000$, you're asking, "To what power must I raise 2 to get 1000?"
- In other words, $2^{what}=1000$
- In this case, $2^{10}=1024$, and since 1000 is less than 1024, the answer will be less than 10.
- Therefore, $\log_2 1000$ is slightly less than 10.

- It is equal to 10 units of time.

Indexing explained

1	1
10	1
20	1
50	2
70	2
80	2
90	3
100	3
200	3
...	...
...	8
...	8
...	8

- Can this search time be improved?
- Of course, yes.

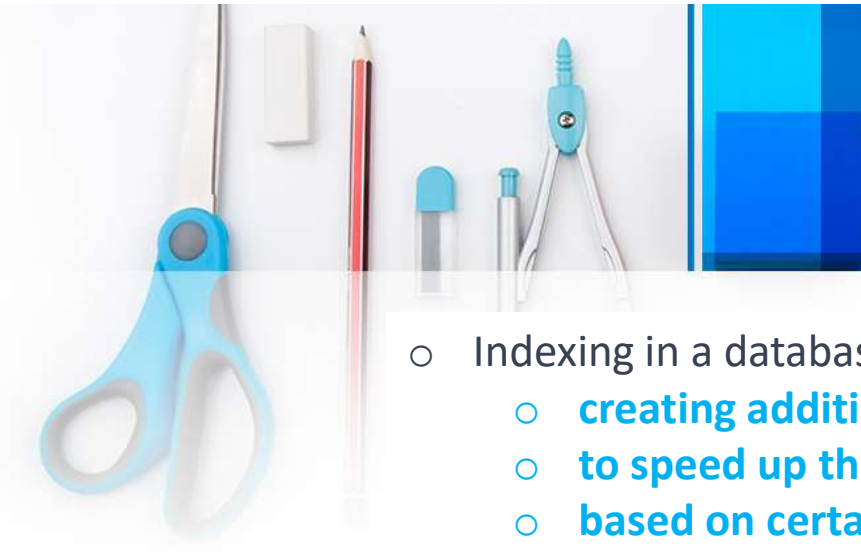
you can apply *indexing* here:

- Maintain a block pointer for each block along with the ordered values used for the previous organization.
- Considering the number of newly indexed records resulted in **8** blocks, you will get an organization like Figure.
- Suppose, you want to search the record **90**. You will result in 3rd iteration.
- Now, $\text{Log}_2 8+1$

In other words, $2^{\text{what}} = 8$? = $2^3 = 8$

So, $\log_2 8 = 3$ and $3+1 = 4$

Therefore, $\log_2 8+1 = 4$



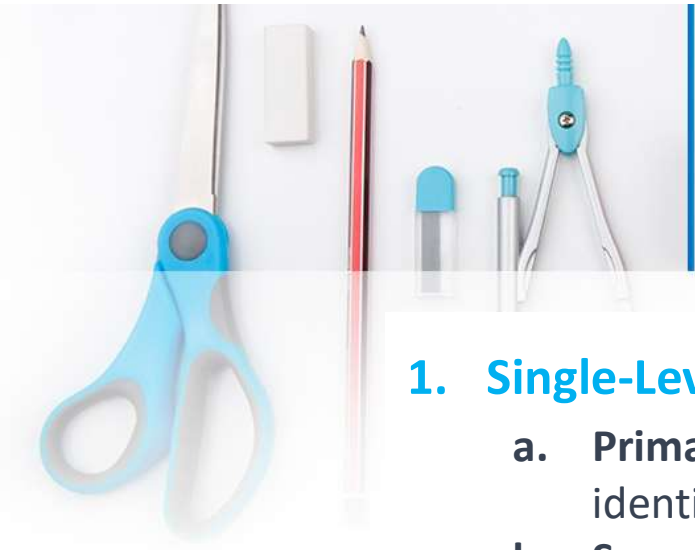
Indexing ?

- Indexing in a database system involves
  - **creating additional data structures**
  - **to speed up the retrieval of data**
  - **based on certain columns.**
  - **Indexing does not physically change the table;**
  - **instead, it creates a separate data structure**
  - **it facilitates faster access to the table's rows.**

Index

Indexing is a technique used to optimize the retrieval of records from a database. Indexes are data structures that improve the speed of data retrieval operations on a database table at the cost of additional writes and storage space. There are different types of indexes:

1. Single-Level Ordered Indexes
2. Multilevel Indexes



Index

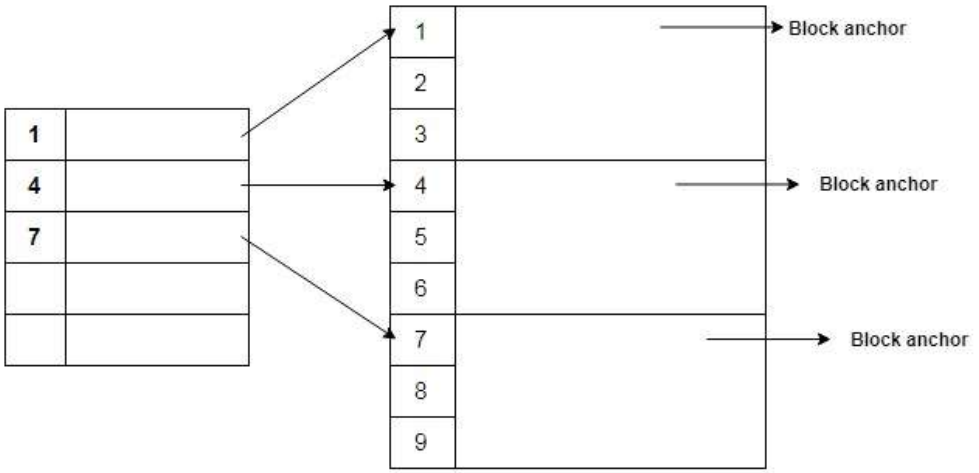
1. Single-Level Ordered Indexes

a. **Primary Index:** It is created on the primary key of a table. It helps in uniquely identifying each record in the table.
b. **Secondary Index:** It is created on a non-primary key column. It speeds up the retrieval of records based on that column but may not be unique.
c. **Clustered Index:** It determines the physical order of data in a table. The data rows in the table are stored in the same order as the index.
d. **Non-Clustered Index:** It does not affect the physical order of the data rows in a table. Instead, it creates a separate structure to store the index data.

1. Single-Level Ordered Indexes - Primary Index

A primary index is an ordered file whose records are of fixed length with two fields:

- The first field is the same as the *primary key* of data file.
- The second field is a pointer to the data block where the primary key is available.



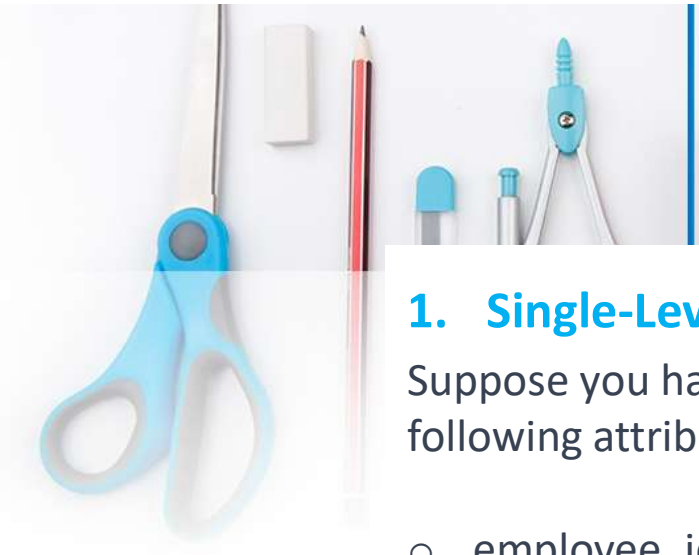
Index

1. Single-Level Ordered Indexes - **Primary Index**

It is created on the primary key of a table. It helps in uniquely identifying each record in the table.

```
CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   employee_name VARCHAR(255),
   department_id INT
);
```

In this example, the **employee_id** column is the primary key, and it automatically creates a primary index on the **employee_id**. The primary index ensures that each **employee_id** is unique and helps in faster retrieval of records based on this key.



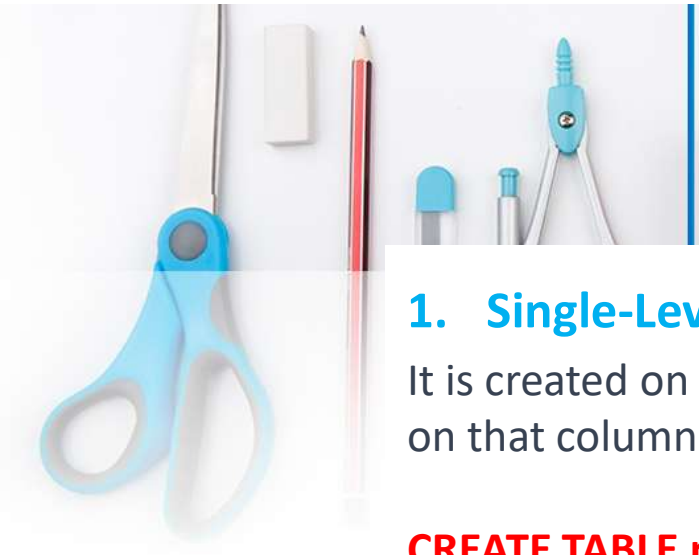
Index

1. Single-Level Ordered Indexes - Secondary Index:

Suppose you have a tabled called Employee in your database. The table has the following attributes:

- employee_id
- employee_name
- employee_department
- employee_salary

employee_id is its primary key. Now, you have already created primary indexing on this table based on employee_id. But while developing an application, you found out that most of the database queries are using the attribute employee_name. In that case, the primary indexing will not help much, and it will be a good practice to maintain separate indexing for all values belonging to employee_name.



Index

1. Single-Level Ordered Indexes - Secondary Index:

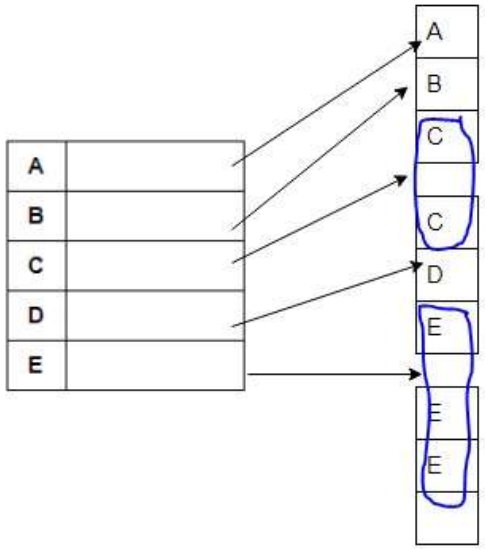
It is created on a non-primary key column. It speeds up the retrieval of records based on that column but may not be unique.

```
CREATE TABLE products (
    product_id INT,
    product_name VARCHAR(255),
    category VARCHAR(50)
);
CREATE INDEX idx_category ON products(category);
```

Here, an index named **idx_category** is created on the category column of the products table. This secondary index allows for faster retrieval of records based on the category column.

1. Single-Level Ordered Indexes - **Clustered Index:**

A Clustering index is created on data file whose file records are physically ordered on a **non-key field** which does not have a distinct value for each record. This field is known as **clustering field** based on which the indexing is performed. Hence the name - clustering index.



Index

1. Single-Level Ordered Indexes - **Clustered Index:**

•It determines the physical order of data in a table. The data rows in the table are stored in the same order as the index. There can be only one clustered index per table.

```
CREATE TABLE customers (
    customer_id INT,
    customer_name VARCHAR(255),
    registration_date DATE
);
CREATE CLUSTERED INDEX idx_registration_date ON customers(registration_date);
```

In this example, a clustered index named **idx_registration_date** is created on the **registration_date** column. The data rows in the **customers** table will be **physically ordered** based on the values in the **registration_date** column.

Index

1. Single-Level Ordered Indexes – Non-Clustered Index:

It does not affect the physical order of the data rows in a table. Instead, it creates a separate structure to store the index data.

```
CREATE TABLE orders (
   order_id INT,
   customer_id INT,
   order_date DATE
);
CREATE NONCLUSTERED INDEX idx_order_date ON orders(order_date);
```

Here, a non-clustered index named **idx_order_date** is created on the **order_date** column of the **orders** table. Unlike a clustered index, the data rows in the table are not physically ordered based on the values in the indexed column.

View Index

In SQL, you can view the existing indexes on a table using various system views or functions provided by the database management system (DBMS).

1. SQL Server:

```
EXEC sp_helpindex 'YourTableName';
```

2. MySQL:

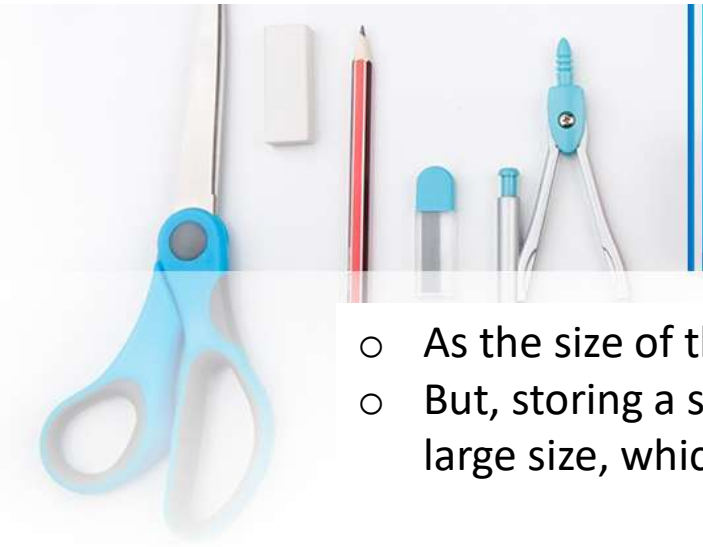
```
SHOW INDEX FROM YourTableName;
```

3. PostgreSQL:

```
SELECT * FROM pg_indexes WHERE tablename = 'YourTableName';
```

4. Oracle:

```
SELECT * FROM USER_INDEXES WHERE TABLE_NAME = 'YourTableName';
```



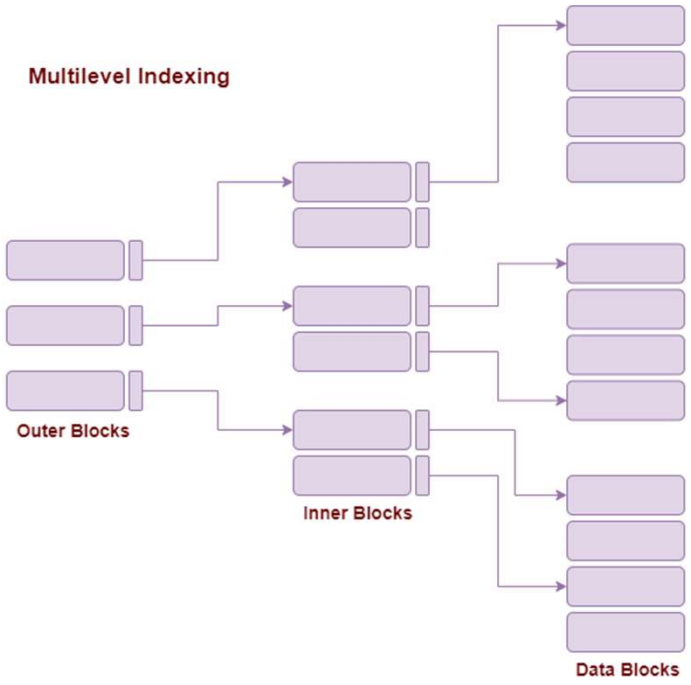
Multilevel Indexing

- As the size of the database increases, the size of the index also increases.
- But, storing a single- level index in main memory may not be practical due to its large size, which requires multiple disk accesses.

- To solve this problem **multilevel indexing** is used.
- This technique breaks down the main block into smaller blocks.
- Each smaller block can be stored in a single block.
- The outer block is further divided into inner blocks, each of which is associated with a data block.
- This approach reduces overhead and makes it easier to store the index in main memory.

Multilevel Indexing

- Multilevel indexes refer to a hierarchical structure of indexes.
- It allows faster data retrieval, reduces disk access, and improves query performance.
- Multilevel indexes are essential in large databases where traditional indexes are nc efficient.
- A multi-level index can be created for any first-level index (primary, secondary, or clustering). It's like a search tree,
- But adding or removing new index entries challenging because every level of the inde is an ordered file.



Trigger

- In SQL, a trigger is a set of instructions that are automatically executed ("triggered") in response to certain events on a particular table or view.
- Events that can activate triggers include INSERT, UPDATE, DELETE operations, or a combination of these.

- **For example**, a trigger can be invoked when a row is inserted into a specified table or when specific table columns are updated in simple words a trigger is a collection of SQL statements with particular names that are stored in system memory.

- Because a trigger cannot be called directly, unlike a stored procedure, it is referred to as a special procedure. A trigger is automatically called whenever a data modification event against a table takes place, which is the main distinction between a trigger and a procedure.

Trigger

In this example, let's say we have a table named Employee:

```
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Salary INT
);

CREATE TABLE EmployeeLog (
   LogID INT PRIMARY KEY, EmployeeID INT,
  OperationType VARCHAR(10),
  OperationTimestamp TIMESTAMP
);
```

Now, we want to create a trigger that automatically updates a log table (EmployeeLog) whenever a new employee is inserted into the Employee table. The log table will store the details of the inserted employee and the timestamp of the operation.

```
CREATE TRIGGER AfterInsertEmployee
AFTER INSERT ON Employee FOR EACH ROW
BEGIN
    INSERT INTO EmployeeLog
   (EmployeeID, OperationType, OperationTimestamp)
    VALUES (NEW.EmployeeID, 'INSERT', NOW());
END;
```

Trigger

Optional

Here's an example of enabling the general query log in MySQL:

- Open your MySQL configuration file (my.cnf or my.ini).
- Add or modify the following lines:
  - [mysqld]
  - general_log = 1
  - general_log_file = /path/to/your/logfile.log
- Replace /path/to/your/logfile.log with the actual path where you want the log file to be stored.
- Restart your MySQL server.

Thank You all!